KRYLOV METHODS FOR THE MINIMUM VARIANCE PORTFOLIO

THOMAS SCHMELZER* AND MICHAEL WOLF†

Abstract. The long-only minimum variance portfolio is typically solved by forming a sample covariance matrix and passing it to a generic quadratic programming solver. This approach is unnecessary: all required operations reduce to matrix-vector products on the returns matrix, and the covariance matrix need never be formed.

Expressed directly in terms of the returns matrix, the problem reduces to a symmetric indefinite KKT system. MINRES and a constraint-eliminated conjugate gradient method solve this system without assembling $X^\top X$ and are 350–385 times faster than CVXPY on a universe of 495 S&P 500 constituents. All methods produce identical portfolio weights; the gain is entirely computational.

Ledoit-Wolf covariance shrinkage acts as a spectral preconditioner for the KKT system: by lifting small eigenvalues it compresses the spectrum and directly reduces iteration counts. On real equity data, where a dominant market factor inflates the condition number, shrinkage halves MINRES iterations and cuts CG by two thirds. The two objectives, reducing statistical estimation error and accelerating the iterative solver, are achieved by the same modification to the spectrum.

1. Introduction. The minimum variance portfolio is one of the most fundamental constructions in quantitative finance. Since [7], it has been standard practice to estimate a covariance matrix from historical returns and solve the resulting quadratic program using a general-purpose optimisation routine.

This workflow is convenient but conceptually misaligned with the structure of the problem. The covariance matrix is not a primitive object; it is an estimate derived from a returns matrix. Once formed, this estimate obscures the underlying data structure and forces the use of dense linear algebra, even though all required operations can be expressed in terms of matrix-vector products on the returns themselves.

Formulated directly in terms of the returns matrix, the problem avoids the covariance matrix entirely and reduces to a symmetric indefinite KKT system solvable by Krylov subspace methods such as MINRES [8].

Generic optimisation frameworks are robust but expensive; matrix-free Krylov methods reduce all operations to repeated applications of X and $X^\top$, avoiding both the $O(n^2)$ storage and the $O(n^2T)$ cost of forming the covariance matrix.

Covariance shrinkage, particularly the estimator of Ledoit and Wolf [5], is usually motivated as a statistical device for high-dimensional estimation. It is also a spectral preconditioner: by lifting small eigenvalues and compressing the spectrum, shrinkage improves the convergence of Krylov methods. **Statistical regularisation and numerical preconditioning act on the same object**, the eigenvalue distribution of the system matrix, driven by different objectives.

The minimum variance problem is formally equivalent to a quadratic programme, but its natural computational representation is linear algebraic: a structured matrix equation that admits fast, scalable solvers without the overhead of a general-purpose optimisation framework. The resulting implementations are faster, more transparent, and better aligned with the statistical structure of the problem, particularly in large-scale settings where forming the covariance matrix is itself the dominant cost.

Krylov methods are standard tools in large-scale numerical linear algebra [3] but have not been systematically applied to portfolio optimisation. The closest related

*Jebel Quant (thomas@jqr.ae). The first author's 2004 diploma thesis at the University of Kaiserslautern [9], supervised by Martin H. Gutknecht (ETH Zürich), developed block extensions of MINRES and SYMMLQ. The present work revisits those same methods in a new setting twenty-two years later.

†Department of Economics, University of Zurich.

work applies GMRES to the MVP problem when the covariance matrix is singular [1]; that formulation still assembles $X^\top X$ explicitly and does not consider preconditioning. General-purpose solvers such as Clarabel [4] handle the long-only constraint natively but require an explicit covariance matrix and cannot exploit the matrix-free structure of the returns representation. The connection between shrinkage intensity and spectral preconditioning does not, to our knowledge, appear in the existing literature.

Sections 2–5 develop the formulation and solvers; Section 6 reports empirical results on both synthetic and S&P 500 data.

2. Problem Formulation. Let $X \in \mathbb{R}^{T\times n}$ be the matrix of daily returns for n assets over T trading days. The long-only minimum variance problem is:

$$\min_w \; w^\top \Sigma w \quad \text{subject to} \quad \mathbf{1}^\top w = 1, \quad w \geq 0$$

where $\Sigma = \frac{1}{T} X^\top X$ is the sample covariance matrix. Since $\Sigma \propto X^\top X$, this is equivalent to:

$$\min_w \; \|Xw\|^2 \quad \text{subject to} \quad \mathbf{1}^\top w = 1, \quad w \geq 0$$

This reframing is key: the objective is now a squared norm of Xw, and we can evaluate $w \mapsto X^\top(Xw)$ using two matrix-vector products, without ever forming $X^\top X$. The factor of $1/T$ in Σ is a positive constant that does not affect the minimiser; it is dropped throughout.

3. The KKT System. Ignoring the inequality constraints for now, the Lagrangian for the equality-constrained problem is:

$$\mathcal{L}(w, \lambda) = \|Xw\|^2 - \lambda(\mathbf{1}^\top w - 1)$$

Setting derivatives to zero gives the stationarity condition $2X^\top Xw = \lambda \mathbf{1}$. Combined with the constraint $\mathbf{1}^\top w = 1$, this yields the augmented KKT system:

$$\begin{pmatrix} 2X^\top X & \mathbf{1} \\ \mathbf{1}^\top & 0 \end{pmatrix} \begin{pmatrix} w \\ \lambda \end{pmatrix} = \begin{pmatrix} \mathbf{0} \\ 1 \end{pmatrix}$$

This $(n+1)\times(n+1)$ system is **symmetric but indefinite**: the zero in the bottom-right corner ensures the matrix has both positive and negative eigenvalues. Standard CG requires positive definiteness and does not apply; MINRES handles the indefinite case directly.

The long-only constraint $w \geq 0$ is handled via an **active set method**: solve the KKT system on the current active set, drop any asset with a negative weight, and repeat. The process terminates in at most n iterations in the worst case.

With Ledoit-Wolf shrinkage, the active set converges rapidly in practice. The unconstrained minimiser satisfies $w^\star \propto \Sigma_{\text{LW}}^{-1}\mathbf{1}$. As $\alpha \to 1$, $\Sigma_{\text{LW}} \to \alpha I$ and $w^\star \to \frac{1}{n}\mathbf{1}$, the equal-weight portfolio with all components strictly positive. Writing $\Sigma_{\text{LW}} = \gamma I + cX^\top X$ and expanding via the Neumann series:

$$\Sigma_{\text{LW}}^{-1}\mathbf{1} = \tfrac{1}{\gamma}\mathbf{1} - \tfrac{c}{\gamma^2} X^\top X \mathbf{1} + O(\gamma^{-3}),$$

every component of $w^\star$ is positive when $\gamma > c\,(X^\top X\mathbf{1})_{\max}$, i.e. when α exceeds a threshold determined by the spectral structure of $X^\top X$. For the LW-optimal α and dense equity returns, this condition is typically satisfied and the first active set solve yields few negative components. Across 50 experiments with n from 100 to 1000 and $T = 2n$, the active set required 2–3 outer iterations in all cases; runtime is dominated by the linear solves.

4. Krylov Solvers for the Indefinite System.

4.1. CVXPY Reference Implementation. Before developing the Krylov solvers we establish a reference solution using CVXPY [2], a domain-specific language for convex optimisation. The minimum variance problem maps directly onto a least-squares objective, which CVXPY dispatches to a general-purpose interior-point solver.

```
import cvxpy as cp

n = R.shape[1]
w = cp.Variable(n)
problem = cp.Problem(
    cp.Minimize(cp.sum_squares(R @ w)),
    [cp.sum(w) == 1, w >= 0]
)
problem.solve(solver=cp.CLARABEL)
weights = w.value
```

The formulation is transparent and correct, and CVXPY handles the long-only constraint natively. We use the Clarabel interior-point solver [4], which dispatches the problem as a second-order cone programme. Its cost is substantial: interior-point methods require $O(n^3)$ work per iteration, and the overhead of problem construction and solver communication adds further latency. At $n = 1000$ this amounts to roughly 5 seconds (Table 1), making CVXPY unsuitable for production use but ideal as a ground-truth reference.

All four methods in this paper (CVXPY, KKT direct, MINRES, and CG) solve the same optimisation problem and produce identical portfolio weights up to numerical tolerance. The comparison is purely computational: the question is not which method finds a better portfolio, but which finds the same portfolio fastest.

4.2. Direct Solve (Baseline). For small n, `numpy.linalg.solve` applied to the explicit KKT matrix is fast and exact. It requires forming $X^\top X$ (an $O(n^2T)$ operation), but for $n \ll T$ this is not the bottleneck.

```
def build_kkt(R):
    """Build the KKT system matrix and RHS for the
    minimum variance problem.

    Constructs the (N+1) x (N+1) indefinite KKT system for:

        min  w^T (R^T R) w
        s.t. sum(w) = 1

    The system has the form:

        [ 2 R^T R   1 ] [ w ]   [ 0 ]
        [ 1^T       0 ] [lam] = [ 1 ]
    """
    n = R.shape[1]
    A = np.zeros((n + 1, n + 1))
    A[:n, :n] = 2 * R.T @ R        # form X^T X explicitly
    A[:n, n] = 1                   # budget constraint column
    A[n, :n] = 1                   # budget constraint row
    b = np.zeros(n + 1);  b[n] = 1
```

```
    return A, b

active = np.ones(n, dtype=bool)
while True:
    A, b = build_kkt(R[:, active])
    sol = np.linalg.solve(A, b)
    w_a = sol[:active.sum()]
    if np.all(w_a >= 0):
        break
    active[np.where(active)[0][w_a < 0]] = False
```

4.3. MINRES. MINRES [8] is a Krylov method for symmetric indefinite systems. It minimises the residual $\|Ax - b\|$ over the Krylov subspace $\mathcal{K}_k(A, b)$. Applied to the KKT system, a matrix-free `LinearOperator` applies the KKT matrix via two matrix-vector products on the returns matrix; $X^\top X$ is never formed.

```
from scipy.sparse.linalg import LinearOperator, minres

def matvec(x):
    out = np.empty(n + 1)
    out[:n] = 2.0 * (R.T @ (R @ x[:n])) + x[n]
    out[n]  = x[:n].sum()
    return out

kkt  = LinearOperator(shape=(n+1, n+1), matvec=matvec)
b    = np.zeros(n + 1);  b[n] = 1.0
w, _ = minres(kkt, b)
```

Convergence depends on the eigenvalue distribution of the KKT matrix. For well-conditioned problems MINRES converges in very few iterations; Section 5 discusses how Ledoit-Wolf shrinkage compresses the spectrum and accelerates convergence in the ill-conditioned case.

4.4. Constraint-Eliminated CG. The equality constraint can be eliminated by parameterising the solution as:

$$w = w_0 + Pv$$

where $w_0 = \frac{1}{n}\mathbf{1}$ is any particular solution satisfying $\mathbf{1}^\top w_0 = 1$, and $P \in \mathbb{R}^{n\times(n-1)}$ is an orthonormal basis for the null space of $\mathbf{1}^\top$, obtained via QR factorisation of $\mathbf{1}$.

Any choice of v gives a feasible w. The reduced unconstrained problem in v is:

$$\min_v \ \|Xw_0 + XPv\|^2$$

with normal equations:

$$(XP)^\top (XP)\, v = -(XP)^\top (Xw_0)$$

The matrix $(XP)^\top(XP)$ is **symmetric positive definite**, so standard CG applies. The matrix-vector product $v \mapsto P^\top X^\top (X(Pv))$ requires four cheap operations and never forms $X^\top X$ or $(XP)^\top(XP)$ explicitly. In the implementation P is never stored: it is applied via an implicit Householder reflector in $O(n)$ time. The full implementation is listed in Appendix A.

5. Ledoit-Wolf Shrinkage. The sample covariance matrix $\frac{1}{T}X^\top X$ is a poor estimator when n is large relative to T: its smallest eigenvalues are severely downward-biased, and the eigenvalue spread (the ratio of largest to smallest) far exceeds that of the true covariance. Ledoit and Wolf [5] propose replacing the sample covariance with a convex combination of the sample estimate and the identity:

$$\Sigma_{\mathrm{LW}} = (1-\alpha)\cdot\frac{X^\top X}{T} + \alpha I$$

The shrinkage intensity $\alpha \in (0,1)$ is chosen analytically to minimise the expected Frobenius distance to the true covariance, requiring no cross-validation. The effect on the spectrum is direct: adding αI lifts every eigenvalue by α, bounding the smallest away from zero and compressing the overall spread. The condition number satisfies:

$$\kappa(\Sigma_{\mathrm{LW}}) = \frac{\lambda_{\max}(\Sigma_{\mathrm{LW}})}{\lambda_{\min}(\Sigma_{\mathrm{LW}})} \le \frac{\lambda_{\max}+\alpha}{\alpha}$$

since $\lambda_{\min}(\Sigma_{\mathrm{LW}}) \ge \alpha$ and $\lambda_{\max}(\Sigma_{\mathrm{LW}}) \le \lambda_{\max}+\alpha$. The condition number is fully controlled by α. Subsequent work by the same authors provides an analytical nonlinear shrinkage formula [6] that applies a different shrinkage coefficient to each sample eigenvalue, achieving a strictly smaller Frobenius loss when n/T is not negligible.

In the implementation, shrinkage is applied by augmenting the return matrix rather than forming Σ_{LW} explicitly. Setting $c = 1-\alpha$ and $\gamma = \alpha\cdot\|\Sigma\|_F/n$, the stacked matrix $\tilde{X} = [\sqrt{c}\,X;\ \sqrt{\gamma}\,I]$ satisfies $\tilde{X}^\top\tilde{X} = c\,X^\top X + \gamma I \propto \Sigma_{\mathrm{LW}}$, so any of the solvers above can be applied to $\tilde{X}$ without modification.

From the perspective of numerical linear algebra, compressing the eigenvalue spectrum of the system matrix is precisely what a preconditioner does. Ledoit-Wolf shrinkage therefore plays a dual role: it reduces statistical estimation error and, as a side effect, accelerates iterative solvers by clustering the eigenvalues. The two motivations are independent but arrive at the same modified matrix.

The identity shrinkage target αI is the simplest choice, but the preconditioning literature suggests sharper alternatives. A factor-model preconditioner, replacing αI with a low-rank-plus-diagonal approximation $BB^\top + D$, would cluster eigenvalues more tightly at the cost of a slightly more expensive application. Incomplete Cholesky factorisation of the shrunk matrix provides a general-purpose option. For rolling-window problems where the covariance changes slowly between periods, a preconditioner built from the previous period's factorisation could be reused with minimal update cost. None of these require forming the covariance matrix explicitly and each is compatible with the matrix-free framework described above.

6. Numerical Results. We present two sets of experiments: a synthetic benchmark with i.i.d. Gaussian returns spanning a range of problem sizes and condition numbers, and a real-data experiment on the S&P 500. All implementations are available in the companion package [10].

Synthetic benchmark. Table 1 reports timings and iteration counts for all four solvers on a problem with $n = 1000$ assets and $T = 2000$ i.i.d. Gaussian returns, run without and with Ledoit-Wolf shrinkage.

On synthetic data, LW shrinkage compresses the eigenvalue spectrum and reduces iteration counts dramatically: MINRES drops from 39 to 16 iterations and CG from 80 to 26. MINRES is the fastest solver in both settings and is roughly 4× faster than KKT direct because it avoids the $O(n^2T)$ cost of assembling $X^\top X$ explicitly.

Method	Time (s)	Iterations	Speedup
Without Ledoit-Wolf shrinkage			
cvxpy (Clarabel)	6.7192	–	1.0x
KKT direct	0.0456	–	147.4x
MINRES	0.0114	39	590.1x
CG (constraint-eliminated)	0.0149	80	451.7x
With Ledoit-Wolf shrinkage			
cvxpy (Clarabel)	4.6214	–	1.0x
KKT direct	0.0473	–	97.6x
MINRES	0.0072	16	643.9x
CG (constraint-eliminated)	0.0083	26	558.3x

TABLE 1

Synthetic benchmark: 1000 assets, 2000 i.i.d. Gaussian returns. Speedup relative to `cvxpy` *within each panel.*

S&P 500 real-data experiment. Table 2 repeats the benchmark on daily log-returns for 495 S&P 500 constituents over the period July 2021–April 2026 (1192 trading days, $T/n \approx 2.4$). Tickers with more than 5% missing observations are excluded; short gaps are forward-filled.

Method	Time (s)	Iterations	Speedup
Without Ledoit-Wolf shrinkage			
cvxpy (Clarabel)	1.5672	–	1.0x
KKT direct	0.0058	–	269.1x
MINRES	0.0059	214	264.5x
CG (constraint-eliminated)	0.0090	387	175.0x
With Ledoit-Wolf shrinkage			
cvxpy (Clarabel)	1.4514	–	1.0x
KKT direct	0.0124	–	117.3x
MINRES	0.0041	105	354.4x
CG (constraint-eliminated)	0.0038	117	384.8x

TABLE 2

S&P 500 universe: 495 assets, 1192 trading days (Jul 2021–Apr 2026). Speedup relative to `cvxpy` *within each panel.*

The real-data results reveal two effects absent from the synthetic benchmark. First, the unshrunk system is substantially harder: MINRES requires 214 iterations and CG 387, compared with 39 and 80 on i.i.d. Gaussian data. The difference reflects the strong factor structure of equity returns: a dominant market factor concentrates variance along a single eigendirection and inflates the condition number far beyond the Gaussian baseline. Without shrinkage, MINRES and KKT direct are nearly indistinguishable in speed (0.0059 s vs. 0.0058 s); the matrix-free advantage is offset by the high iteration count.

Second, Ledoit-Wolf shrinkage matters far more here than on synthetic data. It halves the MINRES count (214 to 105) and cuts CG by two thirds (387 to 117). With shrinkage applied, CG narrowly edges MINRES (0.0038 s vs. 0.0041 s), reversing the

synthetic ranking. CG solves a positive definite reduced system of size $n-1$, and after spectrum compression by shrinkage, its convergence rate catches up to MINRES on the indefinite system of size $n+1$.

A third effect is also visible: KKT direct becomes the *slowest* of the three non-cvxpy solvers when shrinkage is applied (0.0124 s vs. 0.0041–0.0038 s). Shrinkage is implemented by augmenting the return matrix to $\tilde{X} \in \mathbb{R}^{(T+n)\times n}$, making the explicit assembly of $\tilde{X}^\top \tilde{X}$ more expensive than iterating on the original X.

Which solver to use. The choice between MINRES and CG depends on conditioning. Without shrinkage, MINRES is preferable: it handles the indefinite KKT system directly, requires no constraint elimination, and is more robust to high condition numbers; on real equity data without shrinkage, MINRES needs 214 iterations against CG's 387. With Ledoit-Wolf shrinkage applied, the two methods are competitive; on real equity data CG is marginally faster (0.0038 s vs. 0.0041 s) because the constraint-eliminated system of size $n-1$ has slightly cheaper iterations than the KKT system of size $n+1$. CG is also the natural choice when X is sparse or when positive definite structure is required for compatibility with SPD preconditioners. For most practical settings (dense returns with LW shrinkage), either method is acceptable and the difference is small.

Scaling with problem size. Figure 1 extends the synthetic results across a range of problem sizes and shrinkage levels. Panel (a) shows wall-clock time as a function of n (with $T = 2n$ and LW shrinkage throughout). The KKT direct solve scales as $O(n^3)$, dominated by the $n \times n$ linear system, while MINRES and CG grow substantially more slowly, with the gap widening visibly beyond $n \approx 300$.

Panel (b) uses the rank-deficient setting $T = 250 < n = 500$, in which $X^\top X$ has at most T non-zero eigenvalues and the unshrunk sample covariance is singular. This regime is practically relevant: asset universes with more instruments than trading days arise when securities with limited common history are pooled, or when a large cross-section is combined with a short lookback window. Without shrinkage the KKT system is singular and the iterative solvers do not converge; every point in the panel has $\alpha > 0$. The preconditioning effect of shrinkage is strongest precisely here: iteration counts fall steeply as α increases from zero, reaching the practical operating point at moderate α values well below unity.

7. Conclusion. The long-only minimum variance portfolio problem, formulated directly in terms of the returns matrix, is a structured linear algebra problem. Avoiding explicit formation of the sample covariance matrix exposes the KKT system's symmetric indefinite structure and makes Krylov methods directly applicable at scale.

The empirical results, on both synthetic data and 495 S&P 500 constituents over five years, agree. Without shrinkage, real equity returns produce a far harder system than i.i.d. Gaussian data: MINRES needs 214 iterations versus 39, reflecting the dominant market factor that inflates the condition number. Ledoit-Wolf shrinkage [5] compresses the spectrum and reduces iteration counts dramatically in both settings: on synthetic data MINRES drops from 39 to 16 iterations and CG from 80 to 26; on the S&P 500 universe MINRES drops from 214 to 105 and CG from 387 to 117, with the effect more pronounced precisely because the real spectrum is harder to begin with. With shrinkage applied, the Krylov solvers are 350–385× faster than CVXPY and outperform the direct solve by a factor of three, since augmenting the return matrix makes the direct assembly of $\tilde{X}^\top \tilde{X}$ more expensive than iterating.

The shrinkage estimator of Ledoit and Wolf and the preconditioning theory of

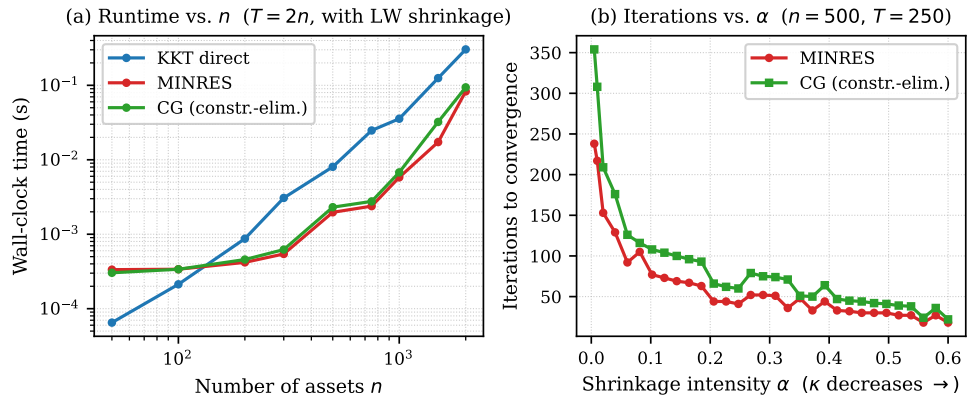

FIG. 1. *Left: wall-clock time vs. number of assets n (log–log scale, $T = 2n$, Ledoit-Wolf shrinkage applied throughout). Right: Krylov iteration counts vs. shrinkage intensity α at fixed $n = 500$, $T = 250$ (rank-deficient baseline). Larger α compresses the eigenvalue spectrum, reducing iterations for both solvers. The condition number κ decreases monotonically as α increases.*

Golub and Van Loan [3] address different problems but act on the same object: the eigenvalue spectrum of the system matrix. This connection, which the two literatures do not usually make explicit, points toward richer preconditioners (factor-model or low-rank structured) as a productive direction for future work.

Acknowledgements. The authors thank Martin H. Gutknecht (ETH Zürich) for supervising the 2004 diploma thesis [9] on block MINRES and SYMMLQ, which motivated the present work.

Appendix A. Constraint-Eliminated CG: Implementation.

```
from scipy.sparse.linalg import LinearOperator, cg

# Implicit null-space basis via Householder reflector
sqrt_n = np.sqrt(float(n))
beta   = 1.0 / (n + sqrt_n)
v0     = 1.0 + sqrt_n

def P_apply(y):                          # P y  (n-1 -> n)
    s = y.sum()
    out = np.empty(n);
    out[0] = -beta * v0 * s;
    out[1:] = y - beta * s
    return out

def PT_apply(u):                         # P^T u  (n -> n-1)
    s = v0 * u[0] + u[1:].sum()
    return u[1:] - beta * s

w0  = np.ones(n) / n            # particular solution
rhs = -PT_apply(R.T @ (R @ w0))

def matvec(y):
    return PT_apply(R.T @ (R @ P_apply(y)))
```

```
A_red = LinearOperator(shape=(n-1, n-1), matvec=matvec)
v, _  = cg(A_red, rhs)
w     = w0 + P_apply(v)                # feasible: 1^T w = 1
```

REFERENCES

[1] I. Bajeux-Besnainou, W. Bandara, and E. Bura, *A Krylov subspace approach to large portfolio optimization*, Journal of Economic Dynamics and Control, 36 (2012), pp. 1688–1699.

[2] S. Diamond and S. Boyd, *CVXPY: A Python-embedded modeling language for convex optimization*, Journal of Machine Learning Research, 17 (2016), pp. 1–5.

[3] G. H. Golub and C. F. Van Loan, *Matrix Computations*, Johns Hopkins University Press, 4th ed., 2013.

[4] P. J. Goulart and Y. Chen, *Clarabel: An interior-point solver for conic optimisation problems with quadratic objectives*, INFORMS Journal on Computing, 36 (2024), pp. 481–499.

[5] O. Ledoit and M. Wolf, *A well-conditioned estimator for large-dimensional covariance matrices*, Journal of Multivariate Analysis, 88 (2004), pp. 365–411.

[6] O. Ledoit and M. Wolf, *Analytical nonlinear shrinkage of large-dimensional covariance matrices*, Annals of Statistics, 48 (2020), pp. 3043–3065.

[7] H. Markowitz, *Portfolio selection*, Journal of Finance, 7 (1952), pp. 77–91.

[8] C. C. Paige and M. A. Saunders, *Solution of sparse indefinite systems of linear equations*, SIAM Journal on Numerical Analysis, 12 (1975), pp. 617–629.

[9] T. Schmelzer, *Block Krylov methods for Hermitian linear systems*, diploma thesis, Department of Mathematics, University of Kaiserslautern, 2004.

[10] T. Schmelzer, *fast-minimum-variance: Solving minimum variance portfolios fast.* https://github.com/Jebel-Quant/fast_minimum_variance, 2026. MIT License.